SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0304	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyze NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	P.Joshna	**Reg. No.:**	192472388
Section / Batch:	Slot B/2024	**Date:**	05/10/2026

Code of Conduct

I P. Joshna(192472388) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Joshna

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.
Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Aim: - To develop a Python-based Regular Expression system to extract candidate information such as name, email, mobile number, technical skills, and years of experience from resume text and identify eligible candidates.

Program Code: -

```
import re
def extract_resume_information(resume):
    name = re.search(r"Name\s*:\s*([A-Za-z ]+)", resume)
    email = re.search(r"\b[\w.-]+@[\w.-]+\.\w+\b", resume)
    mobile = re.search(r"\b[6-9]\d{9}\b", resume)
    experience = re.search(r"(\d+)\s+years?\s+of\s+experience", resume, re.I)
    skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
    found_skills = []
    for skill in skills:
        if re.search(r"\b" + re.escape(skill) + r"\b", resume, re.I):
            found_skills.append(skill)
    candidate_name = name.group(1).strip() if name else "Not Found"
    candidate_email = email.group() if email else "Not Found"
    candidate_mobile = mobile.group() if mobile else "Not Found"
    years = int(experience.group(1)) if experience else 0
    print("\n========== RESUME PROFILE ==========")
    print("Name       :", candidate_name)
    print("Email      :", candidate_email)
    print("Mobile     :", candidate_mobile)
    print("Skills     :", ", ".join(found_skills) if found_skills else "None")
    print("Experience :", years, "years")
    if years >= 2 and "Python" in found_skills:
        print("Eligibility: ELIGIBLE")
```

```
else:
    print("Eligibility: NOT ELIGIBLE")
resume = """
Name: Akshaya G
Email: akshaya@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years of experience
"""
extract_resume_information(resume)
```

Test Case: -

Name: Akshaya G

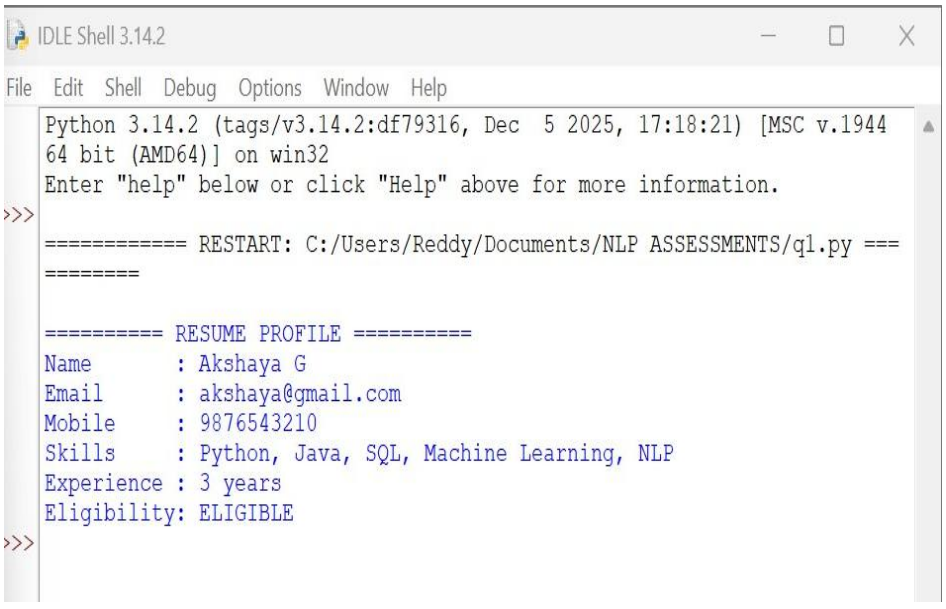
Email: akshaya@gmail.com

Mobile: 9876543210

Skills: Python, Java, SQL, Machine Learning, NLP

Experience: 3 years of experience

Output: -



Result: -The program successfully extracts the required candidate information using Regular Expressions. It also identifies eligible candidates based on experience and Python skill.

Section 2: Regular Expressions – Product Search System

1. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

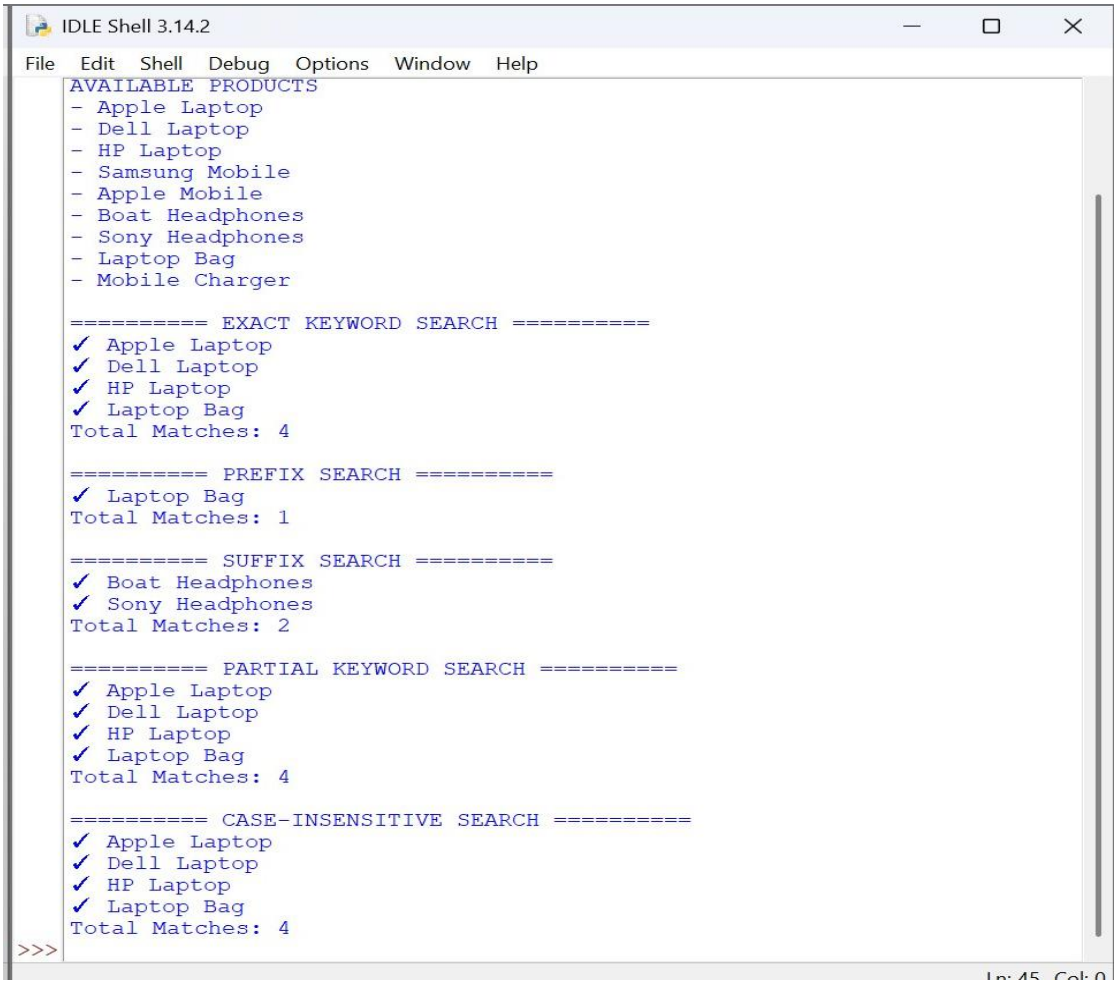
Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Aim: - To develop a Python-based Regular Expression product search system that supports exact, prefix, suffix, partial, and case-insensitive keyword searches and displays the matching products.

Program Code: -

```
import re
products = [
    "Apple Laptop",
    "Dell Laptop",
    "HP Laptop",
    "Samsung Mobile",
    "Apple Mobile",
    "Boat Headphones",
    "Sony Headphones",
    "Laptop Bag",
    "Mobile Charger"
]
def search_products(pattern, title, flags=0):
    matches = []
    for product in products:
        if re.search(pattern, product, flags):
            matches.append(product)
    print("\n==========", title, "==========")
    if matches:
        for product in matches:
            print("✓", product)
    else:
        print("No matching products found.")
    print("Total Matches:", len(matches))
print("AVAILABLE PRODUCTS")
```

```
for product in products:
print("-", product)
search_products(
    r"\bLaptop\b",
    "EXACT KEYWORD SEARCH"
)
search_products(
    r"^Laptop",
    "PREFIX SEARCH"
)
search_products(
    r"Headphones$",
    "SUFFIX SEARCH"
)
search_products(
    r"top",
    "PARTIAL KEYWORD SEARCH"
)
search_products(
    r"laptop",
    "CASE-INSENSITIVE SEARCH",
    re.IGNORECASE
)
```



Result

The program successfully performs exact, prefix, suffix, partial, and case-insensitive product searches. It displays the matching products and total number of matches.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrolment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

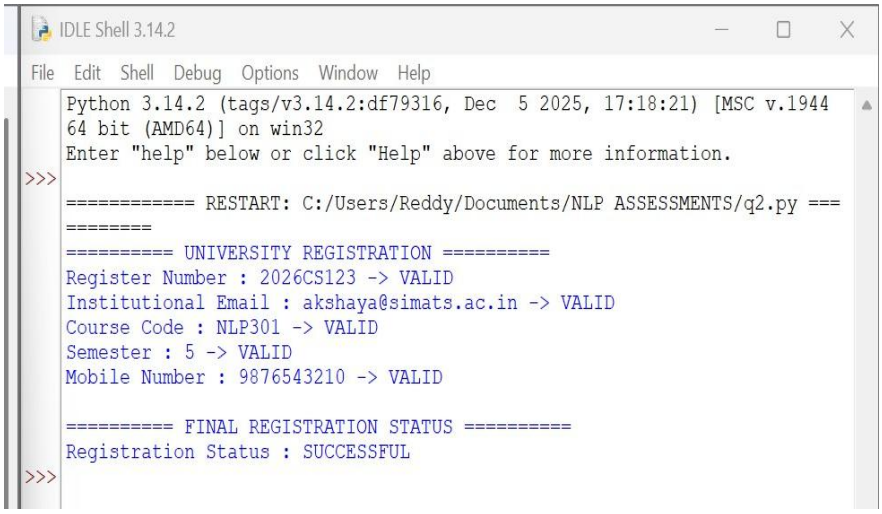
Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Aim: - To develop a Python-based Regular Expression validation system to validate student register number, institutional email, course code, semester, and mobile number and generate the final registration status.

Program: -

```
import re
def validate_register(number):
    return bool(re.fullmatch(r"\d{4}[A-Z]{2}\d{3}", number))
def validate_email(email):
    return bool(re.fullmatch(r"[a-zA-Z0-9._%+-]+@simats\.ac\.in", email))
def validate_course(code):
    return bool(re.fullmatch(r"[A-Z]{2,4}\d{3}", code))
def validate_semester(semester):
    return bool(re.fullmatch(r"[1-8]", semester))
def validate_mobile(mobile):
    return bool(re.fullmatch(r"[6-9]\d{9}", mobile))
def display_result(field, value, status):
    if status:
        print(field, ":", value, "-> VALID")
    else:
        print(field, ":", value, "-> INVALID")
print("========== UNIVERSITY REGISTRATION ==========")
register_no = "2026CS123"
email = "akshaya@simats.ac.in"
course = "NLP301"
semester = "5"
mobile = "9876543210"
register_status = validate_register(register_no)
email_status = validate_email(email)
course_status = validate_course(course)
semester_status = validate_semester(semester)
mobile_status = validate_mobile(mobile)
display_result("Register Number", register_no, register_status)
display_result("Institutional Email", email, email_status)
display_result("Course Code", course, course_status)
display_result("Semester", semester, semester_status)
display_result("Mobile Number", mobile, mobile_status)
print("\n========== FINAL REGISTRATION STATUS ==========")
if all([register_status, email_status, course_status, semester_status, mobile_status]):
    print("Registration Status : SUCCESSFUL")
else:
    print("Registration Status : FAILED")
```

Output: -



Result

The program successfully validates all required student registration details using Regular Expressions. It generates the final registration status based on the validation results.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding g g of Problem	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____